

Autonomous Systems

Decision Making for Mobile Robot Navigation using AlphaBot2

Group 18

Presentation 5

David Gaspar - 106541

Francisco Valério - 106533

Miguel Carneiro - 106218

Pedro Mónico - 106626

ALPHABOT PROBLEMS

Robot Mechanical Problem
Central Sensors Analysis

SOFTWARE IMPLEMENTATION

Script's Logic
Robot Parameters
Script's Output

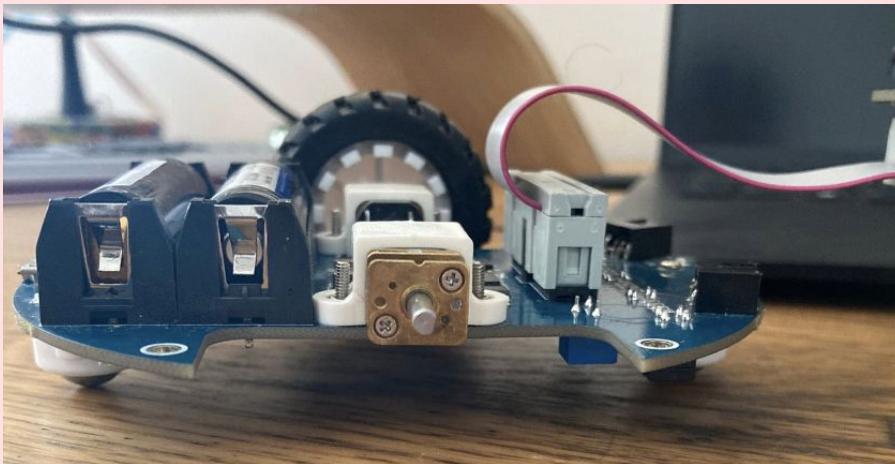
ROBOT CONTROL

Forward Movement
Junction Control

FINAL WEEK: STEPS AND OBJECTIVES

ALPHABOT PROBLEMS

Problem 1



Poor control in angular rotation was caused by screws not being screwed up tightly.

Solution



Dissect the AlphaBot to properly tighten the wheel screws.

Problem 2: Only 2 central sensors reading black at 36mm

Solution: Testing variable line widths to optimize sensor detection



36 mm



38 mm



40 mm

Key Observations

Robot Variability:

- The optimal line width value varies between individual robots.

Detection Threshold:

- Line detection is binary, thresholds fluctuate also varies between individual robots.

ROBOT CONTROL

Forward Movement Control Methodology

Control is carried out using infrared (IR) sensors positioned on the underside of the AlphaBot2

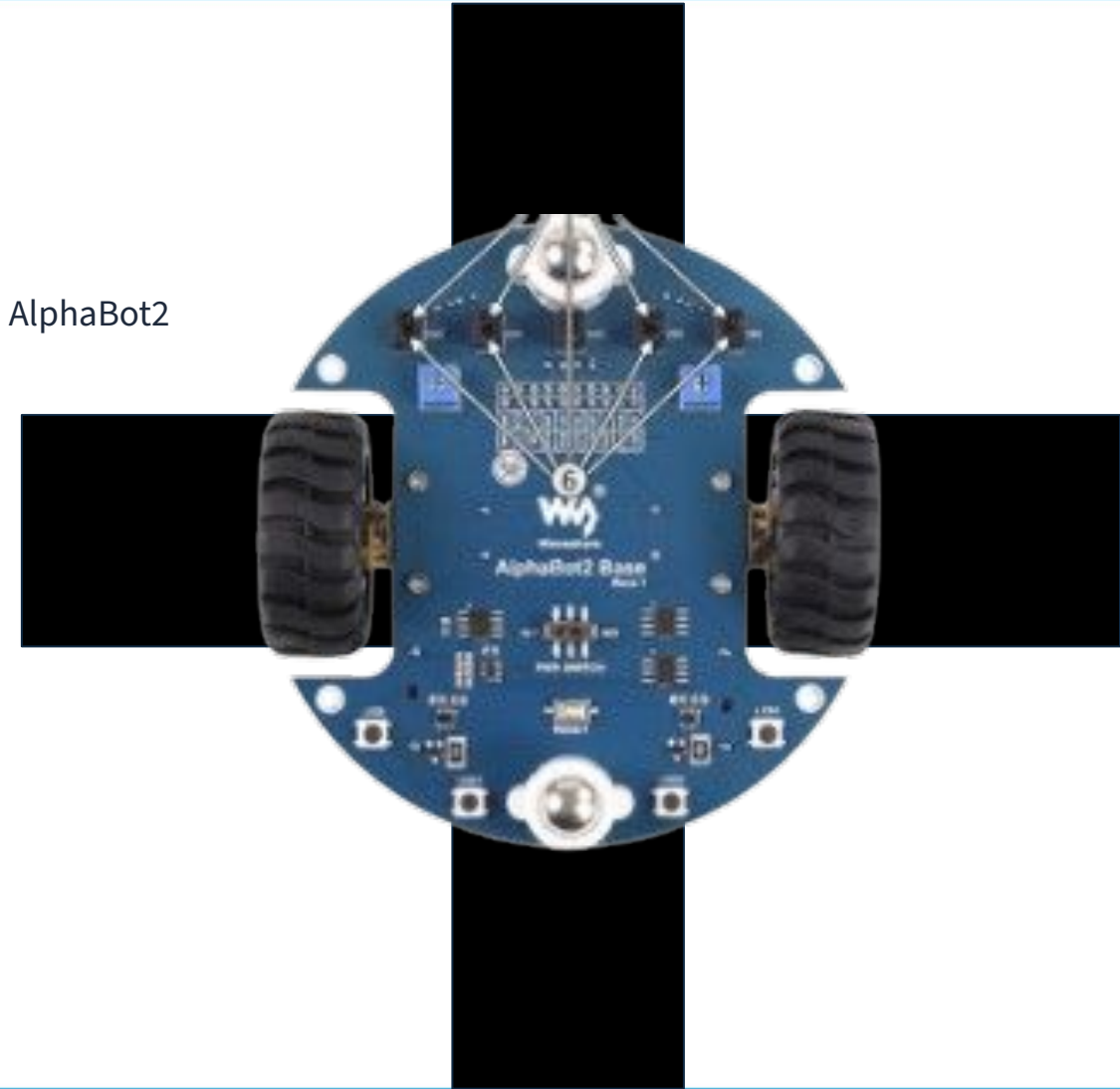


- 01 Ideal State:** The 3 central sensors should detect “black” (line)
- 02 Deviation Detection:** If a sensor detects “white,” the robot identifies a deviation
- 03 Active Correction:** A PD controller adjusts the linear and angular velocity of the robot to recenter the line under the sensors.

Junction Control Methodology

Control is carried out using infrared (IR) sensors positioned on the underside of the AlphaBot2

- 01 Junction Detection:** When all 5 sensors detect "black": [1,1,1,1,1]
- 02 Align Phase:** The robot moves forward until finding the line again
- 03 Search Phase:** The robot rotates until finding a line again [0,1,1,1,0]



SOFTWARE IMPLEMENTATION

main.py**Central Control Loop**

- Runs ROS2 robot control node
- Reads sensors & publishes `cmd_vel`
- Manages FSM: FOLLOW, ALIGN, SEARCH, PAUSE, GOAL
- Initializes Q-learning solver
- Requests actions from the solver at junctions
- Executes actions physically on the robot

world.py**Environment & Mapping**

- Defines the discrete grid world and map
- Stores robot pose and heading
- Handles movement and turning logic
- Checks valid/legal movements
- Provides valid actions for training
- Prints the ASCII world map for debugging/logging

solver.py**Q-Learning Algorithm**

- Implements the Q-learning algorithm
- Trains a Q-table over:
 - State = (`row`, `col`, `heading`)
 - Actions = straight, left, right, turnaround
- Uses rewards + ϵ -greedy exploration
- Returns optimal actions during runtime
- Can save/load trained Q-tables

Input given to the Solver

```
MAP: list[list[int]] = [  
    [0, 1, 0],  
    [0, 0, 0],  
    [1, 0, 0],  
]  
START: tuple[int, int] = (0, 0)           # (row, col)  
START_HEADING: str = "N"                 # N | E | S | W  
GOAL: tuple[int, int] = (2, 2)           # (row, col)
```

```
GOAL_REWARD = 100.0  
ILLEGAL_MOVE_REWARD = -20.0  
ACTION_REWARDS = {  
    ACTION_STRAIGHT: -1.0,  
    ACTION_TURN_RIGHT: -3.0,  
    ACTION_TURN_LEFT: -3.0,  
    ACTION_TURN_AROUND: -6.0,  
}
```

Training Progress Log

```
[solver] episode 50/1000 epsilon=0.778 avg_reward=53.88 avg_steps=11.0 successes=50/50
[solver] episode 100/1000 epsilon=0.606 avg_reward=69.20 avg_steps=8.0 successes=50/50
[solver] episode 150/1000 epsilon=0.471 avg_reward=77.38 avg_steps=6.2 successes=50/50
[solver] episode 200/1000 epsilon=0.367 avg_reward=82.18 avg_steps=5.3 successes=50/50
[solver] episode 250/1000 epsilon=0.286 avg_reward=83.36 avg_steps=5.2 successes=50/50
[solver] episode 300/1000 epsilon=0.222 avg_reward=84.64 avg_steps=4.9 successes=50/50
[solver] episode 350/1000 epsilon=0.173 avg_reward=86.28 avg_steps=4.7 successes=50/50
[solver] episode 400/1000 epsilon=0.135 avg_reward=88.06 avg_steps=4.4 successes=50/50
[solver] episode 450/1000 epsilon=0.105 avg_reward=88.24 avg_steps=4.3 successes=50/50
[solver] episode 500/1000 epsilon=0.082 avg_reward=87.26 avg_steps=4.5 successes=50/50
[solver] episode 550/1000 epsilon=0.063 avg_reward=88.28 avg_steps=4.3 successes=50/50
[solver] episode 600/1000 epsilon=0.050 avg_reward=88.74 avg_steps=4.2 successes=50/50
[solver] episode 650/1000 epsilon=0.050 avg_reward=89.76 avg_steps=4.0 successes=50/50
[solver] episode 700/1000 epsilon=0.050 avg_reward=89.44 avg_steps=4.1 successes=50/50
[solver] episode 750/1000 epsilon=0.050 avg_reward=88.84 avg_steps=4.2 successes=50/50
[solver] episode 800/1000 epsilon=0.050 avg_reward=88.68 avg_steps=4.2 successes=50/50
[solver] episode 850/1000 epsilon=0.050 avg_reward=89.76 avg_steps=4.0 successes=50/50
[solver] episode 900/1000 epsilon=0.050 avg_reward=88.02 avg_steps=4.4 successes=50/50
[solver] episode 950/1000 epsilon=0.050 avg_reward=88.76 avg_steps=4.2 successes=50/50
[solver] episode 1000/1000 epsilon=0.050 avg_reward=89.50 avg_steps=4.1 successes=50/50
[solver] training complete
```

Agent Logic & Grid Map

```
AlphaBot2 cross handler ready (line follow + junctions)
Grid pose: (0,0) heading N goal=(2, 2)
Per-heading action map (S=straight, R=right, L=left, U=u-turn, #=obstacle):
heading N:
  A # A
  S A S
  # S S

heading E:
  R # R
  S S R
  # S L

heading S:
  S # S
  L S S
  # L R

heading W:
  L # L
  A A S
  # R S

----- Map -----
R^  ##  ..
..  ..  ..
##  ..  GG
--- pose (0,0) heading N ---
```

Robot Navigation States (Exploration & Mapping)

```
----- Map -----
R^  ##  ..
..  ..  ..
##  ..  GG
--- pose (0,0) heading N ---
```

```
----- Map -----
R>  ##  ..
..  ..  ..
##  ..  GG
--- pose (0,0) heading E ---
```

```
----- Map -----
Rv  ##  ..
..  ..  ..
##  ..  GG
--- pose (0,0) heading S ---
```

```
----- Map -----
..  ##  ..
Rv  ..  ..
##  ..  GG
--- pose (1,0) heading S ---
```

```
----- Map -----
..  ##  ..
R>  ..  ..
##  ..  GG
--- pose (1,0) heading E ---
```

```
----- Map -----
..  ##  ..
..  R>  ..
##  ..  GG
--- pose (1,1) heading E ---
```

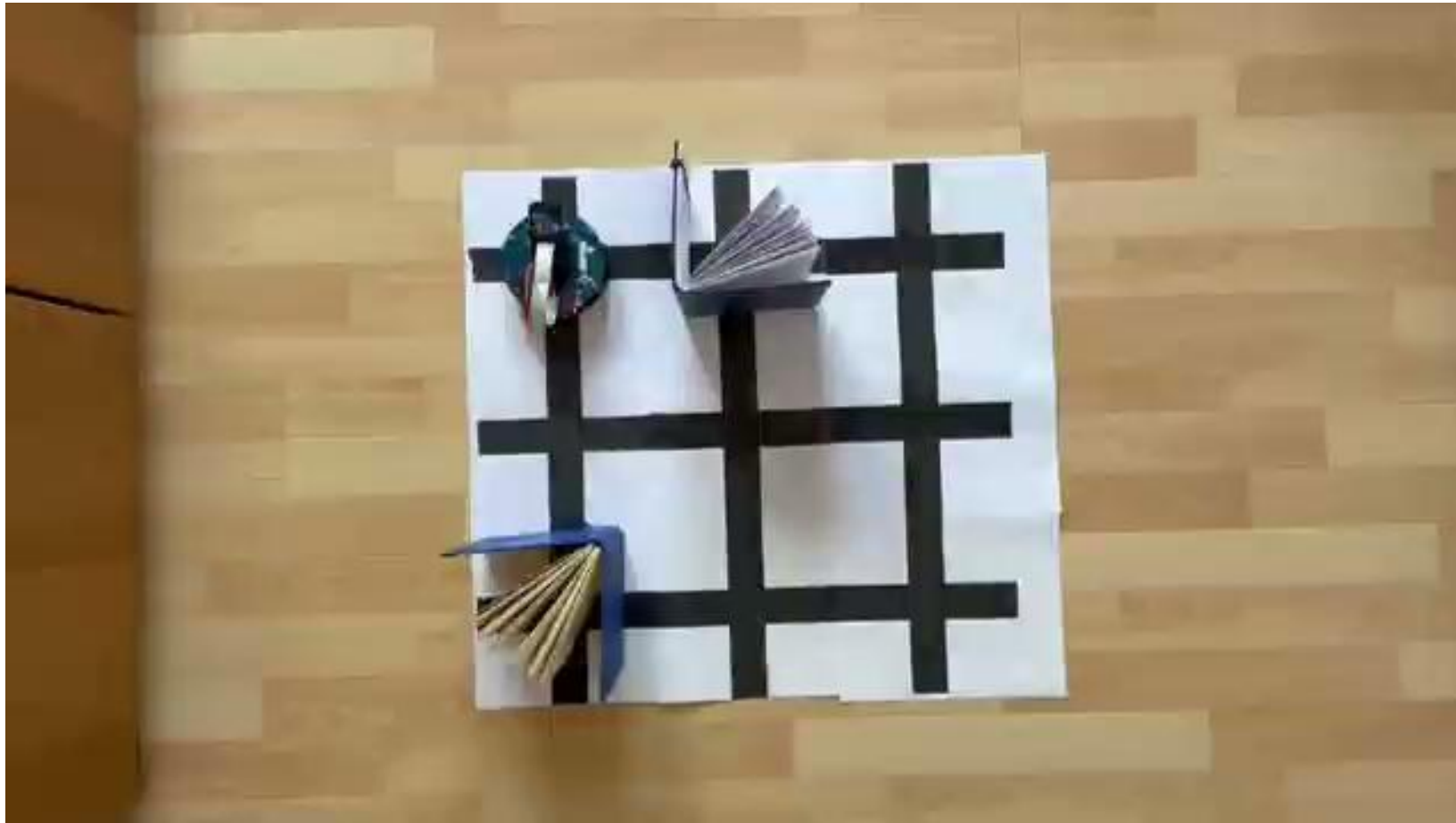
```
----- Map -----
..  ##  ..
..  ..  R>
##  ..  GG
--- pose (1,2) heading E ---
```

```
----- Map -----
..  ##  ..
..  ..  Rv
##  ..  GG
--- pose (1,2) heading S ---
```

Goal Achievement Logic

```
=====
SUCCESS: goal (2, 2) reached - (2,2) heading S
=====
```

```
----- Map -----
..  ##  ..
..  ..  ..
##  ..  Rv
--- pose (2,2) heading S ---
```



NEXT STEPS OF WORKPLAN

TASK	DESCRIPTION
 Online Maze Solver	Implementing a Reinforcement Learning-based maze solver for the AlphaBot2. The robot has no prior map of the maze and learns navigation through trial and error using exploration and exploitation trade-off
 Report	Technical report covering the AlphaBot2 project development, including hardware setup, sensor integration, MDP and RL algorithms and approaches, testing procedures and obtained results.

Thank You!